

Library Management System (.NET NUnit)

Technology: C# (.NET) + NUnit

Name: Tushar Dev Singh

Introduction

This project implements a simple Library Management System with the following features:

- Add new books
- Register borrowers
- Borrow books
- Return books
- View books and borrowers

Unit tests are written using NUnit framework to validate functionality.

Class Implementation

Book.cs:

```
using System;

using System.Collections.Generic;

using System.Text;

namespace LibraryManagementSystem
{
    public class Book
    {
        public string Title { get; set; }

        public string Author { get; set; }

        public string ISBN { get; set; }

        public bool IsBorrowed { get; private set; }

        public Book(string title, string author, string isbn)
        {
            Title = title;

            Author = author;

            ISBN = isbn;

            IsBorrowed = false;
        }

        public void Borrow()
        {
            if (IsBorrowed)
```

```
throw new InvalidOperationException("Book already borrowed");
```

```
IsBorrowed = true;
```

```
}
```

```
public void Return()
```

```
{
```

```
IsBorrowed = false;
```

```
}
```

```
}
```

```
}
```

```
1  using System;
2  using System.Collections.Generic;
3  using System.Text;
4
5
6
7  namespace LibraryManagementSystem
8  {
9      12 references
10     public class Book
11     {
12         2 references 1/1 passing
13         public string Title { get; set; }
14         1 reference
15         public string Author { get; set; }
16         3 references
17         public string ISBN { get; set; }
18         6 references 2/2 passing
19         public bool IsBorrowed { get; private set; }
20
21         2 references 1/1 passing
22         public Book(string title, string author, string isbn)
23         {
24             Title = title;
25             Author = author;
26             ISBN = isbn;
27             IsBorrowed = false;
28         }
29     }
30
31     1 reference
32     public void Borrow()
33     {
34         if (IsBorrowed)
35             throw new InvalidOperationException("Book already borrowed");
36
37         IsBorrowed = true;
38     }
39
40     1 reference
41     public void Return()
42     {
43         IsBorrowed = false;
44     }
45 }
46
47 }
```

55 % No issues found

Borrower.cs:

```
using System.Collections.Generic;

namespace LibraryManagementSystem
{
    public class Borrower
    {
        public string Name { get; set; }

        public string LibraryCardNumber { get; set; }

        public List<Book> BorrowedBooks { get; private set; }

        public Borrower(string name, string cardNumber)
        {
            Name = name;
            LibraryCardNumber = cardNumber;
            BorrowedBooks = new List<Book>();
        }

        public void BorrowBook(Book book)
        {
            BorrowedBooks.Add(book);
        }

        public void ReturnBook(Book book)
        {
            BorrowedBooks.Remove(book);
        }
    }
}
```

LibraryManagementSystemLibraryManagementSystem.Borrower

```
1 using System.Collections.Generic;
2
3 namespace LibraryManagementSystem
4 {
5     8 references
6     public class Borrower
7     {
8         2 references 1/1 passing
9         public string Name { get; set; }
10        3 references
11        public string LibraryCardNumber { get; set; }
12        5 references 2/2 passing
13        public List<Book> BorrowedBooks { get; private set; }
14
15        2 references 1/1 passing
16        public Borrower(string name, string cardNumber)
17        {
18            Name = name;
19            LibraryCardNumber = cardNumber;
20            BorrowedBooks = new List<Book>();
21        }
22
23        1 reference
24        public void BorrowBook(Book book)
25        {
26            BorrowedBooks.Add(book);
27        }
28
29        1 reference
30        public void ReturnBook(Book book)
31        {
32            BorrowedBooks.Remove(book);
33        }
34    }
35 }
```

57 % No issues found

Library.cs:

```
using System;

using System.Collections.Generic;

using System.Text;

using System.Linq;

namespace LibraryManagementSystem
{
    public class Library
    {
        public List<Book> Books { get; private set; } = new List<Book>();

        public List<Borrower> Borrowers { get; private set; } = new List<Borrower>();

        public void AddBook(Book book)
        {
            Books.Add(book);
        }

        public void RegisterBorrower(Borrower borrower)
        {
            Borrowers.Add(borrower);
        }

        public void BorrowBook(string isbn, string cardNumber)
        {
            var book = Books.FirstOrDefault(b => b.ISBN == isbn);

            var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);

            if (book == null || borrower == null)
            {
                throw new Exception("Book or borrower not found");
            }
        }
    }
}
```

```
        book.Borrow();

        borrower.BorrowBook(book);
    }

    public void ReturnBook(string isbn, string cardNumber)
    {
        var book = Books.FirstOrDefault(b => b.ISBN == isbn);
        var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);

        if (book == null || borrower == null)
            throw new Exception("Book or borrower not found");

        book.Return();
        borrower.ReturnBook(book);
    }

    public List<Book> ViewBooks()
    {
        return Books;
    }

    public List<Borrower> ViewBorrowers()
    {
        return Borrowers;
    }
}
```



LibraryManagementSystem



Libra

```
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4 using System.Linq;
5
6 namespace LibraryManagementSystem
7 {
8     2 references
9     public class Library
10     {
11         5 references: 1/1 passing
12         public List<Book> Books { get; private set; } = new List<Book>();
13         1 reference: 1/1 passing
14         public List<Borrower> Borrowers { get; private set; } = new List<Borrower>();
15
16         2 references: 1/1 passing
17         public void AddBook(Book book)
18         {
19             Books.Add(book);
20         }
21
22         2 references: 1/1 passing
23         public void RegisterBorrower(Borrower borrower)
24         {
25             Borrowers.Add(borrower);
26         }
27
28         2 references: 2/2 passing
29         public void BorrowBook(string isbn, string cardNumber)
30         {
31             var book = Books.FirstOrDefault(b => b.ISBN == isbn);
32             var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);
33
34             if (book == null || borrower == null)
35                 throw new Exception("Book or borrower not found");
36
37             book.Borrow();
38             borrower.BorrowBook(book);
39         }
40
41         1 reference: 1/1 passing
42         public void ReturnBook(string isbn, string cardNumber)
43         {
44             var book = Books.FirstOrDefault(b => b.ISBN == isbn);
45             var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber == cardNumber);
46
47             if (book == null || borrower == null)
48                 throw new Exception("Book or borrower not found");
49
50             book.Return();
51             borrower.ReturnBook(book);
52         }
53
54         1 reference: 1/1 passing
55         public List<Book> ViewBooks()
56         {
57             return Books;
58         }
59
60         1 reference: 1/1 passing
61         public List<Borrower> ViewBorrowers()
62         {
63             return Borrowers;
64         }
65     }
66 }
```


Unit Tests (NUnit):

```
using NUnit.Framework;
```

```
using LibraryManagementSystem;
```

```
namespace LibraryManagementSystem.Tests
```

```
{
```

```
    [TestFixture]
```

```
    public class LibraryTests
```

```
    {
```

```
        private Library library;
```

```
        private Book book;
```

```
        private Borrower borrower;
```

```
        [SetUp]
```

```
        public void Setup()
```

```
        {
```

```
            library = new Library();
```

```
            book = new Book("Clean Code", "Robert Martin", "111");
```

```
            borrower = new Borrower("Tushar", "CARD1");
```

```
            library.AddBook(book);
```

```
            library.RegisterBorrower(borrower);
```

```
        }
```

```
        // Add Book Test
```

```
        [Test]
```

```
        public void AddBook_ShouldIncreaseCount()
```

```
        {
```

```
var newBook = new Book("Java", "James", "222");

library.AddBook(newBook);

Assert.That(library.Books.Count, Is.EqualTo(2));
}

//Register Borrower Test
[Test]
public void RegisterBorrower_ShouldIncreaseCount()
{
    var b = new Borrower("Rahul", "CARD2");
    library.RegisterBorrower(b);

    Assert.That(library.Borrowers.Count, Is.EqualTo(2));
}

// Borrow Book Test
[Test]
public void BorrowBook_ShouldMarkAsBorrowed()
{
    library.BorrowBook("111", "CARD1");

    Assert.That(book.IsBorrowed, Is.True);
    Assert.That(borrower.BorrowedBooks.Count, Is.EqualTo(1));
}

// Return Book Test
[Test]
public void ReturnBook_ShouldMakeAvailable()
```

```
{  
  
    library.BorrowBook("111", "CARD1");  
  
    library.ReturnBook("111", "CARD1");  
  
  
    Assert.That(book.IsBorrowed, Is.False);  
  
    Assert.That(borrower.BorrowedBooks.Count, Is.EqualTo(0));  
  
}  
  
  
// View Books Test  
  
[Test]  
public void ViewBooks_ShouldReturnList()  
{  
  
    var books = library.ViewBooks();  
  
  
    Assert.That(books.Count, Is.EqualTo(1));  
  
    Assert.That(books[0].Title, Is.EqualTo("Clean Code"));  
  
}  
  
  
// View Borrowers Test  
  
[Test]  
public void ViewBorrowers_ShouldReturnList()  
{  
  
    var borrowers = library.ViewBorrowers();  
  
  
    Assert.That(borrowers.Count, Is.EqualTo(1));  
  
    Assert.That(borrowers[0].Name, Is.EqualTo("Tushar"));  
  
}  
}  
}
```



LibraryManagementSystem.Tests



```
1 using Xunit;
2 using LibraryManagementSystem;
3
4 namespace LibraryManagementSystem.Tests
5 {
6     [TestFixture]
7     public class LibraryTests
8     {
9         private Library library;
10         private Book book;
11         private Borrower borrower;
12
13         [SetUp]
14         public void Setup()
15         {
16             library = new Library();
17
18             book = new Book("Clean Code", "Robert Martin", "2011");
19             borrower = new Borrower("Teacher", "2022");
20
21             library.AddBook(book);
22             library.RegisterBorrower(borrower);
23         }
24
25         // 10 Add Book Test
26         [Test]
27         #pragma warning disable CS0618 // Type is obsolete
28         public void AddBook_ShouldIncreaseCount()
29         {
30             var newBook = new Book("Clean", "James", "2022");
31             library.AddBook(newBook);
32
33             Assert.That(library.Books.Count, Is.EqualTo(2));
34         }
35
36         // 20 Register Borrower Test
37         [Test]
38         #pragma warning disable CS0618 // Type is obsolete
39         public void RegisterBorrower_ShouldIncreaseCount()
40         {
41             var b = new Borrower("Student", "2022");
42             library.RegisterBorrower(b);
43
44             Assert.That(library.Borrowers.Count, Is.EqualTo(1));
45         }
46
47         // 30 Remove Book Test
48         [Test]
49         #pragma warning disable CS0618 // Type is obsolete
50         public void RemoveBook_ShouldModifyBorrowers()
51         {
52             library.RemoveBook("2011", "2022");
53
54             Assert.That(book.IsBorrowed, Is.True);
55             Assert.That(borrower.BorrowedBooks.Count, Is.EqualTo(1));
56         }
57
58         // 40 Return Book Test
59         [Test]
60         #pragma warning disable CS0618 // Type is obsolete
61         public void ReturnBook_ShouldModifyBorrower()
62         {
63             library.ReturnBook("2011", "2022");
64             library.ReturnBook("2011", "2022");
65
66             Assert.That(book.IsBorrowed, Is.False);
67             Assert.That(borrower.BorrowedBooks.Count, Is.EqualTo(0));
68         }
69
70         // 50 View Books Test
71         [Test]
72         #pragma warning disable CS0618 // Type is obsolete
73         public void ViewBooks_ShouldReturnList()
74         {
75             var books = library.ViewBooks();
76
77             Assert.That(books.Count, Is.EqualTo(1));
78             Assert.That(books[0].Title, Is.EqualTo("Clean Code"));
79         }
80
81         // 60 View Borrowers Test
82         [Test]
83         #pragma warning disable CS0618 // Type is obsolete
84         public void ViewBorrowers_ShouldReturnList()
85         {
86             var borrowers = library.ViewBorrowers();
87
88             Assert.That(borrowers.Count, Is.EqualTo(1));
89             Assert.That(borrowers[0].Name, Is.EqualTo("Teacher"));
90         }
91     }
92 }
```

26 %



No issues found



Test Results:

The screenshot displays the Visual Studio IDE with the following components:

- Test Explorer:** Shows a table of test results for 'LibraryManagementSystem.Tests'. The table has columns for Test, Duration, Traits, and Error Message. The results show 6 tests passed in 6 ms.
- Group Summary:** Displays the overall test results for the group 'LibraryManagementSystem.Tests'. It shows 6 tests in the group, a total duration of 6 ms, and 6 passed outcomes.
- Solution Explorer:** Shows the project structure with 'LibraryManagementSystem.Tests' selected.
- Output Window:** Displays build logs. The logs indicate that the build started at 02:42 AM, succeeded, and completed at 02:42 AM, taking 00.068 seconds.

Test	Duration	Traits	Error Message
LibraryManagementSystem.Tests (6)	6 ms		

```
Build started at 02:42 AM...
===== Build: 0 succeeded, 0 failed, 2 up-to-date, 0 skipped =====
----- Build completed at 02:42 AM and took 00.068 seconds -----
```